

Лабораторная работа №4: Документирование кода и подготовка к интеграции

1. Теоретические сведения

1.1. Стандарты оформления кода (PEP 8)

PEP 8 (Python Enhancement Proposal 8) — это официальное руководство по стилю кодирования для Python. Оно описывает рекомендации по тому, как писать код на Python, чтобы он был максимально читаемым и единообразным. Следование PEP 8 не является обязательным для работы кода, но крайне важно для командной разработки, поддержки и понимания кода другими разработчиками [1].

Основные принципы PEP 8:

- **Отступы:** Используйте 4 пробела для каждого уровня отступа. Не используйте табы.
- **Длина строки:** Ограничивайте длину строки 79 символами (для кода) и 72 символами (для docstrings и комментариев).
- **Пустые строки:** Используйте две пустые строки для разделения функций верхнего уровня и классов. Используйте одну пустую строку для разделения методов внутри класса.
- **Импорты:** Импорты должны быть в начале файла, каждый импорт на отдельной строке. Сначала стандартные библиотеки, затем сторонние, затем собственные модули. Разделяйте группы импортов пустой строкой.
- **Именованное:**
 - Переменные, функции, методы: `snake_case` (например, `my_variable`, `calculate_total`).
 - Классы: `CamelCase` (например, `MyClass`, `UserService`).
 - Константы: `ALL_CAPS` (например, `MAX_CONNECTIONS`).
 - Приватные методы/атрибуты: начинаются с одного подчеркивания (например, `_private_method`).
 - Магические методы: начинаются и заканчиваются двумя подчеркиваниями (например, `__init__`).
- **Пробелы:**

- Избегайте лишних пробелов внутри скобок, перед запятыми, двоеточиями.
- Используйте пробелы вокруг операторов (например, `a = b + c`).

Пример кода до и после применения PEP 8:

```
# Плохой стиль (не PEP 8)
import os,sys

class myclass:
    def __init__(self,name,value):
        self.name=name
        self.value=value
    def GET_NAME(self):
        return self.name

def calculate(a,b):
    return a*b+ (b/a)

MY_CONSTANT=100
```

```
# Хороший стиль (PEP 8)
import os
import sys

class MyClass:
    def __init__(self, name, value):
        self.name = name
        self.value = value

    def get_name(self):
        return self.name

def calculate(a, b):
    return a * b + (b / a)

MY_CONSTANT = 100
```

1.2. Автоматическая проверка стиля: `flake8` и `black`

Ручная проверка соответствия PEP 8 может быть утомительной и подверженной ошибкам. Для автоматизации этого процесса существуют специальные инструменты.

1.2.1. `flake8`

`flake8` — это утилита, которая объединяет несколько инструментов для проверки стиля и статического анализа кода: `pycodestyle` (проверяет соответствие PEP 8), `pyflakes` (проверяет логические ошибки, такие как неиспользуемые импорты или переменные) и `mccabe` (проверяет цикломатическую сложность кода) [2].

Установка `flake8`:

```
pip install flake8
```

Использование `flake8`:

Просто запустите `flake8` в корне вашего проекта:

```
flake8 .
```

`flake8` выведет список ошибок и предупреждений с указанием файла, строки, позиции и кода ошибки (например, `E501` для слишком длинной строки).

1.2.2. `black`

`black` — это бескомпромиссный форматор кода Python. Он автоматически переформатирует ваш код в соответствии с PEP 8 и другими рекомендациями, не оставляя места для споров о стиле. Его философия — "форматирование без компромиссов" [3].

Установка `black`:

```
pip install black
```

Использование `black`:

Для форматирования файла или директории:

```
black your_module.py  
black your_project_directory/
```

Для проверки без изменения файлов:

```
black --check your_project_directory/
```

`black` часто используется вместе с `flake8`: `black` форматирует код, а `flake8` проверяет на оставшиеся ошибки (например, неиспользуемые импорты, которые `black` не исправляет).

1.3. Аннотации типов (Type Hinting) и статический анализ с `mypy`

Python — это динамически типизированный язык, что означает, что типы переменных определяются во время выполнения. Однако, начиная с Python 3.5, появилась возможность добавлять **аннотации типов (Type Hinting)**, которые позволяют указывать ожидаемые типы данных для переменных, аргументов функций и возвращаемых значений. Это значительно улучшает читаемость кода, помогает предотвращать ошибки и облегчает работу IDE [4].

Основные возможности аннотаций типов:

- **Аргументы функций и возвращаемые значения:**

```
def greeting(name: str) -> str:
    return "Hello, " + name
```

- **Переменные:**

```
age: int = 30
name: str
```

- **Сложные типы (из модуля `typing`):**

- `List[int]`, `Dict[str, int]`, `Tuple[str, ...]`, `Set[float]`
- `Optional[str]` (может быть `str` или `None`)
- `Union[str, int]` (может быть `str` или `int`)
- `Any` (любой тип)
- `Callable[[Arg1Type, Arg2Type], ReturnType]` (для функций)

Пример использования аннотаций типов:

```
from typing import List, Dict, Optional, Union

def process_data(data: List[Dict[str, Union[str, int]]], threshold: Optional[int]) -> List[str]:
    result: List[str] = []
    for item in data:
        if threshold is None or item.get("value", 0) > threshold:
            result.append(item.get("name", "Unknown"))
    return result

# Пример использования
my_data = [
    {"name": "Item A", "value": 10},
    {"name": "Item B", "value": 25},
    {"name": "Item C", "value": 5},
]
```

```
filtered_names = process_data(my_data, threshold=15)
print(filtered_names) # ['Item B']
```

1.3.1. Статический анализатор `myru`

Аннотации типов сами по себе не влияют на выполнение кода. Для их проверки используются **статические анализаторы типов**, такие как `myru`. `myru` анализирует ваш код без его выполнения и проверяет соответствие типов, указанных в аннотациях [5].

Установка `myru`:

```
pip install myru
```

Использование `myru`:

Запустите `myru` для вашего файла или проекта:

```
myru your_module.py
myru your_project_directory/
```

`myru` выведет ошибки, если обнаружит несоответствие типов, что поможет выявить потенциальные баги до запуска кода.

```
# Пример кода с ошибкой типа для myru

def add_numbers(a: int, b: int) -> int:
    return a + b

result: str = add_numbers(1, 2) # myru выдаст ошибку: Incompatible types in ass:
```

Использование аннотаций типов в сочетании с `myru` значительно повышает надежность и поддерживаемость Python-кода, особенно в больших проектах.

1.4. Документирование кода (Docstrings)

Docstrings (строки документации) — это строковые литералы, которые появляются сразу после определения функции, метода, класса или модуля. Они используются для документирования кода и предоставляют краткое, но исчерпывающее описание того, что делает объект, его аргументы, возвращаемые значения и возможные исключения. Docstrings доступны во время выполнения через атрибут `__doc__` объекта [6].

Основные форматы Docstrings:

Существует несколько популярных форматов docstrings, каждый из которых имеет свои особенности. Важно выбрать один формат и придерживаться его в рамках проекта.

1.4.1. Формат Google (Google Style Docstrings)

Один из наиболее читаемых и широко используемых форматов. Он использует секции для описания аргументов, возвращаемых значений и исключений.

```
def example_function(param1: str, param2: int) -> bool:
    """Краткое описание функции.

    Более подробное описание функции, если необходимо. Может занимать несколько

    Args:
        param1 (str): Описание первого параметра.
        param2 (int): Описание второго параметра.

    Returns:
        bool: Описание возвращаемого значения (True, если успешно, False иначе).

    Raises:
        ValueError: Если param2 отрицательное.

    Examples:
        >>> example_function("test", 10)
        True
        >>> example_function("error", -1)
        Traceback (most recent call last):
          ...
        ValueError: param2 не может быть отрицательным
    """
    if param2 < 0:
        raise ValueError("param2 не может быть отрицательным")
    return len(param1) > param2
```

1.4.2. Формат NumPy (NumPy Style Docstrings)

Популярен в научных и численных библиотеках. Использует похожую структуру с секциями, но с немного другим синтаксисом.

```
def calculate_mean(data: list[float]) -> float:
    """Вычисляет среднее арифметическое списка чисел.
```

Parameters

`data : list[float]`

Список чисел, для которых нужно вычислить среднее.

Returns

`float`

Среднее арифметическое значение.

Raises

`ValueError`

Если входной список пуст.

Examples

`>>> calculate_mean([1.0, 2.0, 3.0])``2.0``>>> calculate_mean([])`

Traceback (most recent call last):

`...``ValueError: Список данных не может быть пустым``"""``if not data:` `raise ValueError("Список данных не может быть пустым")``return sum(data) / len(data)`

1.4.3. Формат reStructuredText (Sphinx Style Docstrings)

Используется Sphinx для генерации документации. Часто применяется с директивами `:param:`, `:type:`, `:returns:`, `:rtype:`, `:raises:`.

```
def process_text(text: str, upper_case: bool = False) -> str:
```

```
    """Обрабатывает текстовую строку.
```

```
    :param text: Входная текстовая строка.
```

```
    :type text: str
```

```
    :param upper_case: Если True, строка будет преобразована в верхний регистр.
```

```
    :type upper_case: bool, optional
```

```
    :returns: Обработанная строка.
```

```
    :rtype: str
```

```
    :raises TypeError: Если `text` не является строкой.
```

```
    """
```

```
    if not isinstance(text, str):
```

```
        raise TypeError("Входной текст должен быть строкой.")
```

```
if upper_case:
    return text.upper()
return text
```

1.5. Автоматическая генерация документации с Sphinx

Sphinx — это мощный генератор документации, который преобразует исходные файлы (обычно написанные в формате reStructuredText или Markdown) в различные выходные форматы, такие как HTML, PDF (через LaTeX), ePub и другие. Он широко используется для документирования проектов на Python, включая сам Python, NumPy, Django и многие другие [7].

Основные возможности Sphinx:

- **Автоматическая генерация из Docstrings:** Sphinx может извлекать docstrings из вашего Python-кода и автоматически включать их в документацию.
- **Поддержка reStructuredText:** Мощный язык разметки для создания структурированной документации.
- **Расширяемость:** Множество расширений для поддержки различных функций (например, математические формулы, графики, ссылки на GitHub).
- **Темы:** Возможность настраивать внешний вид документации.

Процесс создания документации с Sphinx:

1. Установка Sphinx:

```
pip install sphinx sphinx-rtd-theme
```

`sphinx-rtd-theme` — это популярная тема оформления.

2. **Инициализация проекта Sphinx:** Перейдите в корневую директорию вашего проекта и создайте поддиректорию для документации (например, `docs`). Затем внутри нее запустите `sphinx-quickstart`:

```
mkdir docs
cd docs
sphinx-quickstart
```

`sphinx-quickstart` задаст несколько вопросов о вашем проекте (название, автор, версия и т.д.) и создаст базовую структуру файлов:

- `conf.py`: Основной конфигурационный файл Sphinx.
- `index.rst`: Главный файл документации.

- `_static/`, `_templates/`: Директории для статических файлов и шаблонов.

3. **Настройка `conf.py`**: Для того чтобы Sphinx мог найти ваш Python-код и извлечь из него docstrings, необходимо добавить путь к вашему проекту в `sys.path` в файле `conf.py`. Также нужно включить расширения `sphinx.ext.autodoc` и `sphinx.ext.napoleon` (если вы используете Google или NumPy формат docstrings) и указать тему.

```
# conf.py
import os
import sys
sys.path.insert(0, os.path.abspath("..")) # Путь к корневой директории ваш

project = 'My Python Project'
copyright = '2023, Your Name'
author = 'Your Name'
release = '0.1.0'

extensions = [
    'sphinx.ext.autodoc', # Для автоматического извлечения docstrings
    'sphinx.ext.napoleon', # Для поддержки Google/NumPy style docstrings
    'sphinx.ext.viewcode', # Для просмотра исходного кода
    'sphinx.ext.todo', # Для поддержки заметок TODO
]

html_theme = 'sphinx_rtd_theme'
```

4. **Создание файлов `.rst` для модулей**: Используйте утилиту `sphinx-apidoc` для автоматической генерации `.rst` файлов для ваших модулей и пакетов. Запустите ее из директории `docs`:

```
sphinx-apidoc -o . ..
```

Эта команда создаст файлы `your_module.rst` и `modules.rst` (или `packages.rst`).

Пример содержимого `your_module.rst`:

```
your_module module
=====

.. automodule:: your_module
   :members:
```

```
:undoc-members:  
:show-inheritance:
```

5. **Включение `.rst` файлов в `index.rst`**: Добавьте ссылки на сгенерированные `.rst` файлы в `index.rst`:

```
.. toctree::  
    :maxdepth: 2  
    :caption: Contents:  
  
    modules
```

6. **Генерация документации**: Из директории `docs` выполните команду:

```
make html
```

Это сгенерирует HTML-документацию в директории `_build/html`.
Откройте `_build/html/index.html` в браузере, чтобы посмотреть результат.

Sphinx — это мощный инструмент, который позволяет создавать профессиональную и легко поддерживаемую документацию для ваших Python-проектов, что является критически важным для любого серьезного программного продукта.

1.6. Управление зависимостями: `requirements.txt` и `pyproject.toml`

В любом реальном Python-проекте используются сторонние библиотеки и пакеты. Эффективное управление этими зависимостями критически важно для воспроизводимости окружения, совместной разработки и развертывания приложений.

1.6.1. `requirements.txt`

Файл `requirements.txt` — это самый простой и распространенный способ зафиксировать список зависимостей проекта. Он представляет собой текстовый файл, в котором каждая строка содержит название пакета и, опционально, его версию [8].

Пример `requirements.txt`:

```
requests==2.28.1  
beautifulsoup4>=4.11.1,<5.0.0  
pytest~=7.2.0
```

```
flake8
black
mypy
sphinx
sphinx-rtd-theme
```

Создание `requirements.txt`:

Вы можете вручную перечислить все зависимости или использовать `pip freeze` для генерации файла из текущего виртуального окружения:

```
pip freeze > requirements.txt
```

Установка зависимостей из `requirements.txt`:

```
pip install -r requirements.txt
```

Преимущества `requirements.txt`:

- Простота использования.
- Широкая поддержка.

Недостатки `requirements.txt`:

- Не управляет транзитивными зависимостями (зависимостями зависимостей) явно.
- Может привести к проблемам с версиями, если не указаны точные версии.

1.6.2. `pyproject.toml` и `Poetry` / `Rye`

`pyproject.toml` — это стандартный файл конфигурации для проектов Python, введенный в PEP 518 и расширенный в PEP 621. Он предназначен для унификации конфигурации различных инструментов сборки и управления проектами [9].

Современные инструменты управления зависимостями, такие как **Poetry** и **Rye**, используют `pyproject.toml` для декларативного определения зависимостей, метаданных проекта и других настроек. Они решают проблемы `requirements.txt`, обеспечивая детерминированную установку зависимостей и управление виртуальными окружениями.

Пример `pyproject.toml` (с использованием синтаксиса `Poetry`):

```
[tool.poetry]
name = "my-python-project"
version = "0.1.0"
description = "A simple Python project for demonstration."
authors = ["Your Name <you@example.com>"]
readme = "README.md"
```

```
[tool.poetry.dependencies]
python = "^3.9"
requests = "^2.28.1"
beautifulsoup4 = "^4.11.1"

[tool.poetry.group.dev.dependencies]
pytest = "^7.2.0"
flake8 = "^6.0.0"
black = "^23.1.0"
mypy = "^1.0.0"
sphinx = "^6.1.3"
sphinx-rtd-theme = "^1.2.0"

[build-system]
requires = ["poetry-core>=1.0.0"]
build-backend = "poetry.core.masonry.api"
```

Основные преимущества инструментов на базе `pyproject.toml` (Poetry, Rye):

- **Декларативное управление зависимостями:** Все зависимости (включая транзитивные) фиксируются в файле `poetry.lock` (для Poetry), обеспечивая воспроизводимость.
- **Изолированные виртуальные окружения:** Автоматически создают и управляют виртуальными окружениями для каждого проекта.
- **Управление метаданными проекта:** Позволяют легко определять название, версию, авторов и другие метаданные проекта.
- **Публикация пакетов:** Упрощают процесс создания и публикации пакетов на PyPI.

Установка Poetry:

```
curl -sSL https://install.python-poetry.org | python3 -
```

Использование Poetry (примеры):

- Инициализация проекта: `poetry init`
- Добавление зависимости: `poetry add requests`
- Установка зависимостей: `poetry install`
- Запуск команд в виртуальном окружении: `poetry run python your_script.py`

Выбор между `requirements.txt` и инструментами на базе `pyproject.toml` зависит от сложности проекта и предпочтений команды. Для простых скриптов

`requirements.txt` может быть достаточным, но для более крупных и сложных проектов `Poetry` или `Rye` предлагают значительно более мощные и надежные решения.

✓ 2. Практические задания

Задание №1: Приведение кода к стандартам PEP 8 и форматирование

Возьмите любой Python-файл из ваших предыдущих лабораторных работ (например, парсер из ЛР №1), который не был специально отформатирован.

Требования:

- Установите `flake8` и `black`.
- Запустите `flake8` на выбранном файле и проанализируйте выданные ошибки и предупреждения.
- Используйте `black` для автоматического форматирования файла.
- Повторно запустите `flake8` и убедитесь, что большинство ошибок стиля исчезло. Исправьте оставшиеся ошибки вручную, если они не были устранены `black` (например, неиспользуемые импорты).
- Опишите, какие изменения внес `black` и какие ошибки пришлось исправлять вручную.

Задание №2: Документирование кода и генерация документации с Sphinx

Выберите один из ваших модулей или классов из предыдущих лабораторных работ (например, API-клиент из ЛР №1).

Требования:

- Добавьте docstrings в формате Google Style (или NumPy Style, или reStructuredText Style) ко всем функциям, методам и классам в выбранном модуле.
- Установите Sphinx и `sphinx-rtd-theme`.
- Инициализируйте проект Sphinx в отдельной директории `docs`.
- Настройте `conf.py` для автоматического извлечения docstrings из вашего модуля (используйте `sphinx.ext.autodoc` и `sphinx.ext.napoleon`, если выбрали Google/NumPy Style).

- Сгенерируйте HTML-документацию и убедитесь, что она корректно отображает информацию о вашем коде.
- Предоставьте ссылку на сгенерированную документацию (если вы развернули ее локально, то путь к `index.html`).

Задание №3. Контрольные вопросы

1. Что такое PEP 8 и почему важно следовать его рекомендациям?
2. Для чего используются `flake8` и `black`? В чем их основные отличия и как они дополняют друг друга?
3. Что такое `mypy` и как он используется для статического анализа кода?
4. Объясните концепцию Docstrings. Где они размещаются и для чего используются?
5. Что такое Sphinx и для чего он нужен в Python-разработке?
6. Какие шаги необходимо выполнить для генерации HTML-документации с помощью Sphinx?
7. Для чего нужен файл `conf.py` в проекте Sphinx?
8. В чем основные преимущества `Poetry` (или `Rye`) по сравнению с `requirements.txt`?
9. Почему важно управлять зависимостями в Python-проектах?